| TXURDINAGA CIFP LHII | Kurtsoa / *Curso* | Data / *Fecha* | Ebal. / *Eval.* | Kalifikazioa / *Calificación* |
|---|---|---|---|---|
| | 2025-26 | 03/10/2025 | 1ª EVAL | |
| Taldea / *Grupo* | Modulua / *Módulo* | IE / *RA* | | |
| DWC | 2DW3 | | | |
| Izen-abizenak / *Nombre y apellidos* | | | | |

**EXAMEN**

- Este examen es de tipo procedimental.
- Tiene un valor total de 10 puntos.
- Si la propuesta del examen se resuelve de forma correcta se le aplica el valor especificado en cada apartado.
- Descarga el código fuente necesario para hacer el examen desde Moodle.
- Una vez terminado el examen, comprime los archivos y súbelos a la tarea de Moodle.

**Enunciado:**

La empresa **Txurdi-Netflix** quiere lanzar una versión cliente en Javascript de su aplicación web de películas. Para ello, se te pide implementar las siguientes funcionalidades. **Todos los apartados son independientes.**

**Ejercicio 1. Arrays: (2 puntos)**

En el archivo **peliculas.js** dispones de un array inicial con varios objetos literales de películas, cada uno con: titulo, director, anio, categoría, valoracion.

Realiza las siguientes operaciones:

1. Crea una función "mayusculas" que muestre por consola **todos los títulos de las películas en mayúsculas.**
2. Crea una función "peliculasPorCategoria(categoria)" que devuelva un **nuevo array solo con las películas de la categoría indicada**.
3. Crea una función buscarPorTitulo(titulo) que devuelva **todas las películas cuyo título contenga el texto indicado** (sin distinguir mayúsculas/minúsculas). Por ejemplo, debería poder extraer la película "Mufasa: The Lion King" solamente escribiendo "Lion".
4. Crea una función "mediaValoraciones()" que devuelva un valor con la **media de las valoraciones de todas las películas.**

   *Nota: Las funciones 2, 3 y 4 deben devolver un valor o un array no mostrar un valor o un array por consola.*

| **Total ejercicio – 1 Arrays** | **2** |
|---|---|
| a. Ha creado la función **mayúsculas** y muestra todos los títulos por consola en mayúsculas correctamente. | 0.3 |
| b. Ha creado la función **películasPorCategoria(categoría)** y devuelve un nuevo array solo con las películas de la categoría indicada. | 0.5 |
| c. Ha creado la función **buscarPorTitulo(titulo)** y devuelve las películas cuyo título contenga el texto indicado (búsqueda literal). | 0.4 |
| d. En la función **buscarPorTitulo**, además, no tiene en cuenta mayúsculas ni minúsculas. | 0.4 |
| e. Ha creado la función **mediaValoraciones** y devuelve el valor con la media de las valoraciones de todas las películas. | 0.4 |

*Nota: si las funciones que deben devolver un valor o un array muestran los valores por consola desde la función se descontarán 0,2 puntos por función. Para comprobar el resultado ejecuta algo como* `console.log(buscarPorTitulo("Dune"))`

**Ejercicio 2. Objetos: (1.5 puntos)**

Crea una clase Pelicula con las propiedades: titulo, director, anio, categoría, valoracion.

1. Define un **constructor** que inicialice estas propiedades.
2. Implementa un método **mostrar()** que devuelva una cadena con este formato: "Matrix (1999), dirigida por Wachowski, en la categoría de Ciencia Ficción".
3. Crea **dos objetos** de la clase Pelicula, guárdalos en un array y recórrelo mostrando el resultado de mostrar() por consola.

| **Total ejercicio – 2 Objetos** | **1.5** |
|---|---|
| a. Ha creado la clase **Pelicula** con sus atributos siguiendo la nomenclatura recomendada. | 0.3 |
| b. Ha definido el constructor que inicializa las propiedades correctamente. | 0.3 |
| c. Ha implementado el método **mostrar()** que devuelve una cadena con el formato especificado (ej: "Matrix (1999), dirigida por Wachowski, en la categoría de Ciencia Ficción"). | 0.5 |
| d. Ha creado dos objetos de la clase Pelicula correctamente. | 0.1 |
| e. Ha guardado en un array los objetos de la clase película. | 0.1 |
| f. Ha mostrado el resultado de la función **mostrar()** por consola. | 0.2 |

**Ejercicio 3. Eventos y DOM (2,5 puntos)**

En el archivo index.html hay un listado <ul id="listaPeliculas"> vacío y un botón con id "btnMostrar".

1. Asocia al botón un evento que, al pulsarlo, recorra el array de películas y muestre cada título como un <li> dentro de la lista.

   Se valorará positivamente que utilices los **eventos del DOM** con los que creas los elementos y no innerHTML.

2. Añade un efecto: si el usuario pasa el ratón por encima de un título de película, este se pondrá en **negrita**; al salir, volverá al estilo normal.

| **Total ejercicio – 3 Eventos y DOM** | **2.5** |
|---|---|
| a. Ha asociado al botón "**btnMostrar**" un evento al pulsarlo. | 0.3 |
| b. Recorre el array de películas y muestra cada título como un **<li>** dentro de la lista con id="listaPeliculas". | 0.5 |
| c. Ha implementado la creación de elementos con funciones del DOM, sin usar innerHTML o similares. | 0.5 |
| c. Ha implementado el efecto de poner el texto en negrita al pasar el ratón por encima del título. | 0.6 |
| d. Ha implementado el efecto de volver al estilo normal al salir del título. | 0.6 |

**Ejercicio 4. Validación de formularios (2 puntos)**

En el archivo formulario.html existe un formulario con los campos: título, director, anio, categoría.

Y un botón de tipo submit con id=”validar” que permite validar el formulario.

Implementa en **validacion.js** la validación con las siguientes condiciones:

1. título y director: deben tener entre **3 y 50 caracteres alfabéticos**. Usa una **expresión regular** que lo compruebe.
2. anio: debe ser un número de **4 dígitos** comprendido entre 1900 y 2099.
3. categoría: **no puede estar vacía.**

Si algún campo no cumple los requisitos, debes mostrar los errores: o bien todos juntos en un único mensaje, o cada uno de manera independiente.

Si todo es correcto, el formulario no se enviará, pero se mostrará el mensaje "Película validada correctamente" en el .

Nota: Si lo necesitas para la posterior validación con Javascript, puedes modificar el formulario para añadir restricciones en las etiquetas HTML.

***Nota: no se valorará validación realizada únicamente con HTML5.***

| **Total ejercicio – 4 Validación de formularios y Eventos** | **2.0** |
|---|---|
| a. Ha realizado la validación de **título** y **director** correctamente (entre 3 y 50 caracteres alfabéticos), utilizando una expresión regular. | 0.6 |
| b. Ha realizado la validación de **anio** correctamente (número de 4 dígitos comprendido entre 1900 y 2099). | 0.5 |
| c. Ha realizado la validación de **categoría** correctamente (no puede estar vacía). | 0.2 |
| d. Si todos los campos son correctos, evita el envío del formulario | 0.2 |
| e. Si la validación es correcta, muestra el mensaje "**Película validada correctamente**" en el . | 0.3 |
| f. Implementa la lógica para mostrar los errores de validación (todos juntos o cada uno de manera independiente). | 0.2 |

**Ejercicio 5. Web Storage (2 puntos)**

En usuario.html hay un formulario con un botón “Login”.

1. Al enviar el formulario con el primer botón, guarda el nombre del usuario en **localStorage**.
2. Si ya existe un usuario guardado, en lugar del formulario se mostrará el mensaje: "Bienvenido de nuevo, <usuario>" y un botón **Cerrar sesión**.
3. Al pulsar **Cerrar sesión**, se eliminará el usuario de localStorage y se volverá a mostrar el formulario.

| **Total ejercicio – 5 Web Storage** | **2.0** |
|---|---|
| a. Al enviar el formulario con el botón "Login", guarda el nombre del usuario en localStorage. | 0.4 |
| b. Si ya existe un usuario guardado, **oculta** el formulario. | 0.3 |
| c. Si ya existe un usuario guardado, muestra el mensaje: "**Bienvenido de nuevo,** ". | 0.4 |
| d. Si ya existe un usuario guardado, muestra el botón "**Cerrar sesión**" | 0.3 |
| e. Al pulsar "**Cerrar sesió**n", se elimina el usuario de localStorage. | 0.3 |
| f. Al pulsar "**Cerrar sesión**", se vuelve a mostrar el formulario. | 0.3 |